Task 2 – Image Inclusion & Resource Management in Compose

Priyanka K

1. Introduction

This task focuses on understanding how images are included in Jetpack Compose projects, how the `Image` composable works, and why proper resource management (such as image density and optimization) is important in Android development.

2. What is the Image Composable?

The `Image` composable in Jetpack Compose is used to display images in the user interface. It can load images from drawable resources using `painterResource()`, or from other sources such as URLs.

Example from implementation:

```
Image(
    painter = painterResource(id = R.drawable.androidparty),
    contentDescription = "Birthday Image"
)
```

In this project, the `Image` composable is used to display the image `androidparty` as part of the birthday card screen.

3. Why Image Density Matters in Android

Android devices have different screen densities (mdpi, hdpi, xhdpi, etc.). If image density is not handled properly:

- Images may appear blurry on high-density screens.
- Images may appear too large or too small.
- App performance may reduce due to oversized images.

Using appropriate drawable folders (such as `drawable-nodpi`) prevents Android from automatically scaling the image and ensures consistent appearance across devices. Proper density management improves visual quality and performance.

4. Best Practices for Adding Images in Compose

1. Use Clear Naming Conventions

Images should have meaningful names such as `androidparty.png` instead of generic names like `image1.png`. This improves project readability and maintainability.

2. Optimize Image Size

Avoid using very large resolution images (e.g., 4000x3000 px) when not necessary. Compress images and resize them appropriately to reduce memory usage and improve performance.

5. Visual Storyboard (Screen Evolution)

Frame 1 – Placeholder Text Only

This frame shows the UI before adding any image. The layout uses:

- `Column` for vertical arrangement
- `Text` composables for displaying message

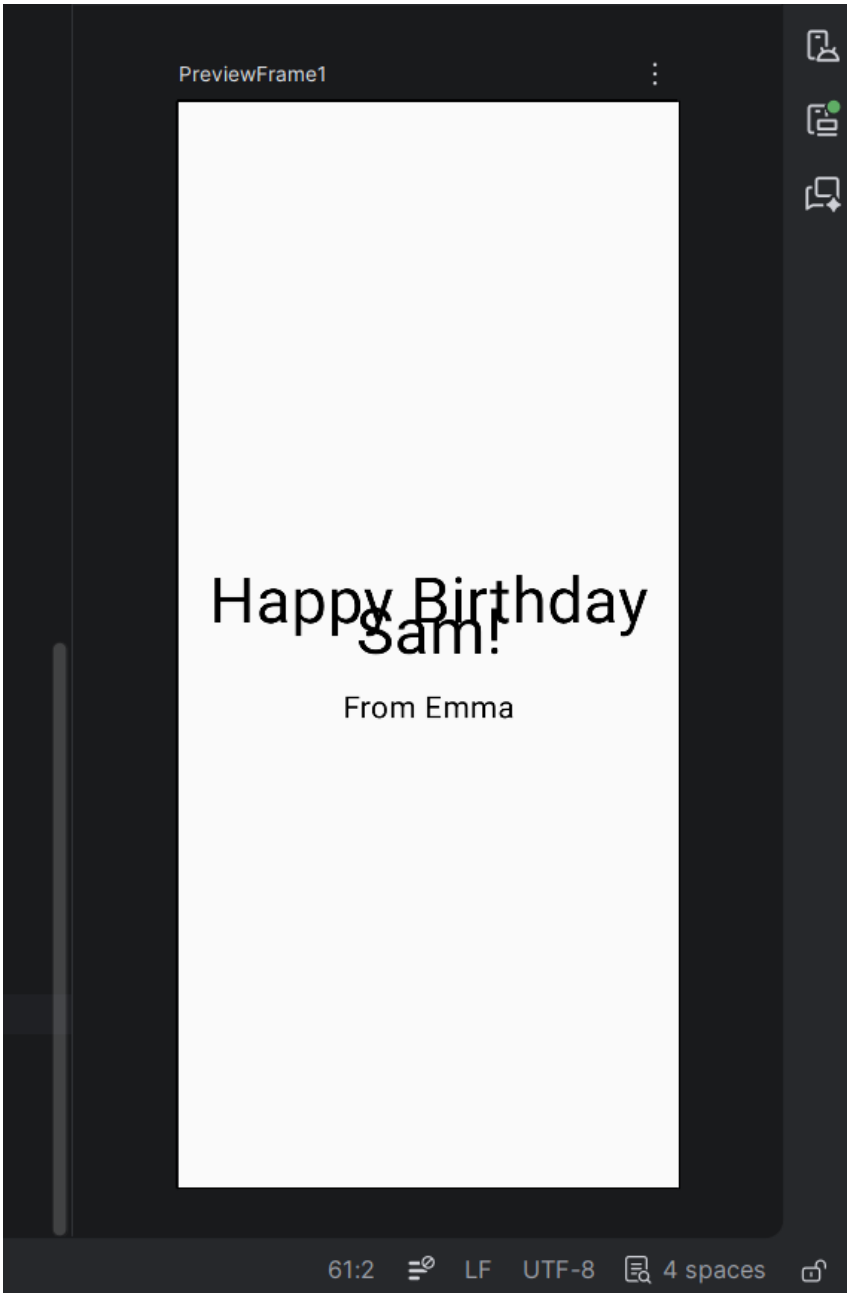

Figure 1: Frame 1: Screen with Placeholder Text Only

Frame 2 – Image Added at Top Right

In this frame, the image is added using:

- `Box` for layered layout
- `Image` composable
- Alignment set to `Alignment.TopEnd`

The image is smaller and positioned at the top-right corner to show how adding an image changes layout structure.

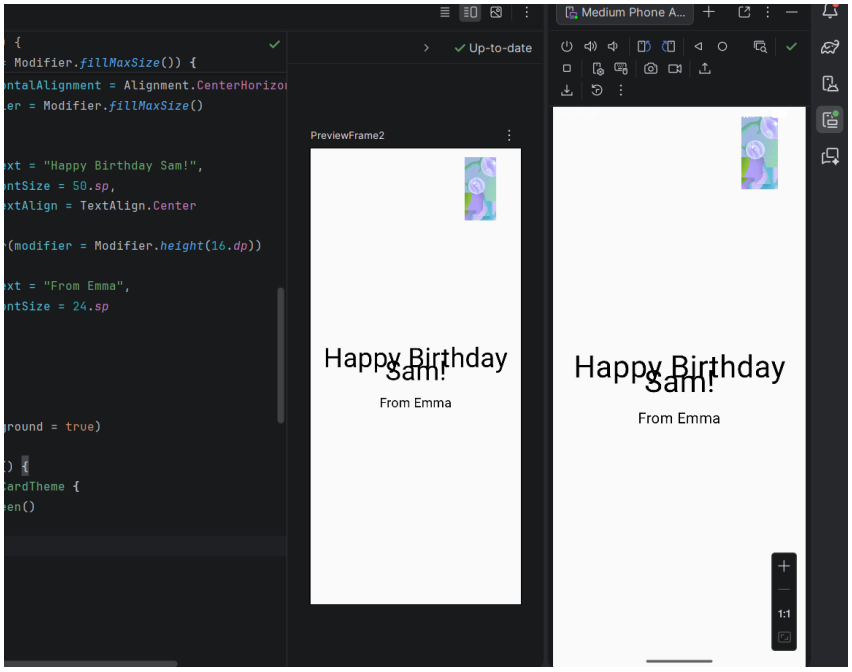

Figure 2: Frame 2: Image Added at Top Right

Frame 3 – Image Adjusted (Full Screen with Crop)

In this frame, the image is modified to:

- Fill the entire screen using `Modifier.fillMaxSize()`
- Use `ContentScale.Crop` for proper scaling

This demonstrates how image placement and scaling affect overall UI layout.

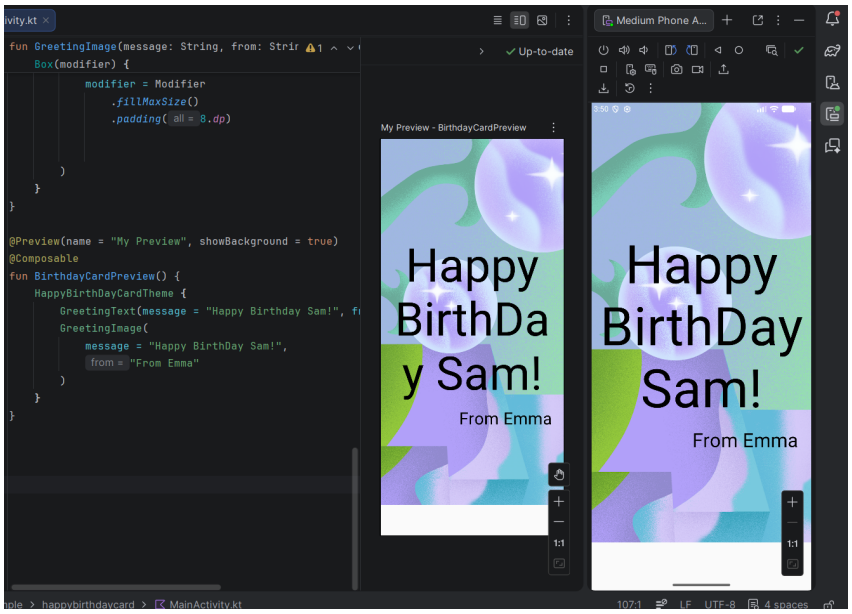

Figure 3: Frame 3: Image Adjusted with Full Screen and Crop

6. Reflection

Images introduce several challenges in UI design. Large image files can reduce performance and increase memory usage. Incorrect scaling can distort layout appearance across devices. Accessibility must also be considered by providing meaningful content descriptions. Planning image placement and optimization before coding helps create efficient and visually consistent Compose applications.